# VisionGuard: AI vs Real Image Classification & Deployment

Assignment 2 Final Project Report • Machine Learning Operations (MLOps)

| | | | |
|---|---|---|---|
| **Student Name** | Shayan Qadir | **Roll Number** | 23L-0790 |
| **Course** | MLOps (Spring 2026) | **Project Identifier** | 23L0790_A2_Ops |
| **GitHub Repo** | github.com/Shayan-Qadir/23L-0790_A2_MLOPS | **DagsHub Project** | dagshub.com/Shayan-Qadir/23L0790_A2_Ops |
| **Public API URL** | 23l-0790a2mlops.vercel.app/api/predict | **Live Web App** | 23l-0790a2mlops.vercel.app |
| **Graded Commit** | 1667840 | **Cloud Deployment** | Vercel Serverless (Pathway B) / OCI VM (Pathway A) |

**AI Tool Disclosure Statement:** I used Google Antigravity (Gemini 3.8 Flash assistant) as an interactive pair programming tool for syntax assistance, Flask HTML/CSS template boilerplate, and pytest test stubs. All data cleaning, pipeline engineering, model training, MLflow tracking, DVC setup, Docker/Compose design, serverless optimization, error analysis, and report writing were executed and verified by me.

## Task 1: Data Understanding and Versioning

**Environment & Windows Setup:** A Python 3.13.5 virtual environment was created for this project. Since TensorFlow 2.11+, native GPU wheels are no longer supported on Windows without WSL2 or TensorFlow-DirectML. Because of this, CPU execution was used and confirmed via tf.config.list_physical_devices('GPU') returning empty. All models and pipelines were designed to run efficiently on CPU.

**Dataset Audit:** I wrote src/data/dataset_audit.py to inspect all raw images in data/raw. Each image was checked for file corruption using PIL, hashed with SHA-256 to detect duplicates, and inspected for dimensions and color modes.

| Metric | Overall | AiArtData (Class 1) | RealArt (Class 0) |
|---|---|---|---|
| Image Count | 975 (100%) | 539 (55.3%) | 436 (44.7%) |
| Corrupt Files | 0 (0%) | 0 | 0 |
| Exact Duplicates | 2 files (1 pair) | image33.jpg & image34.jpg | 0 |
| Width (Min / Med / Max) | 183 / 1024 / 8495 px | 225 / 1024 / 6000 px | 183 / 920 / 8495 px |
| Height (Min / Med / Max) | 135 / 768 / 7091 px | 135 / 800 / 4001 px | 158 / 720 / 7091 px |
| File Size (Min / Med / Max) | 11 KB / 165 KB / 9.9 MB | 11 KB / 195 KB / 5.5 MB | 11 KB / 141 KB / 9.9 MB |
| Color Modes | RGB: 896, RGBA: 49, P: 23, L: 7 | RGB: 489, RGBA: 35, P: 14, L: 1 | RGB: 407, RGBA: 14, P: 9, L: 6 |

**EDA Findings & Pipeline Decisions:**

• *Duplicate Removal:* image34.jpg had an identical SHA-256 hash to image33.jpg in AiArtData; it was dropped to prevent data leakage.
• *Resolution Variance:* Dimensions range from 183×135 to 8495×7091. All images were standardized to 224×224 in the pipeline.
• *Color Modes:* 49 RGBA, 23 Palette, and 7 Grayscale images exist. The pipeline forces conversion to 3-channel RGB to avoid shape mismatch.
• *Payload Size:* File sizes reach up to 9.9MB, which justified adding a 5MB limit in Flask to prevent memory exhaustion.

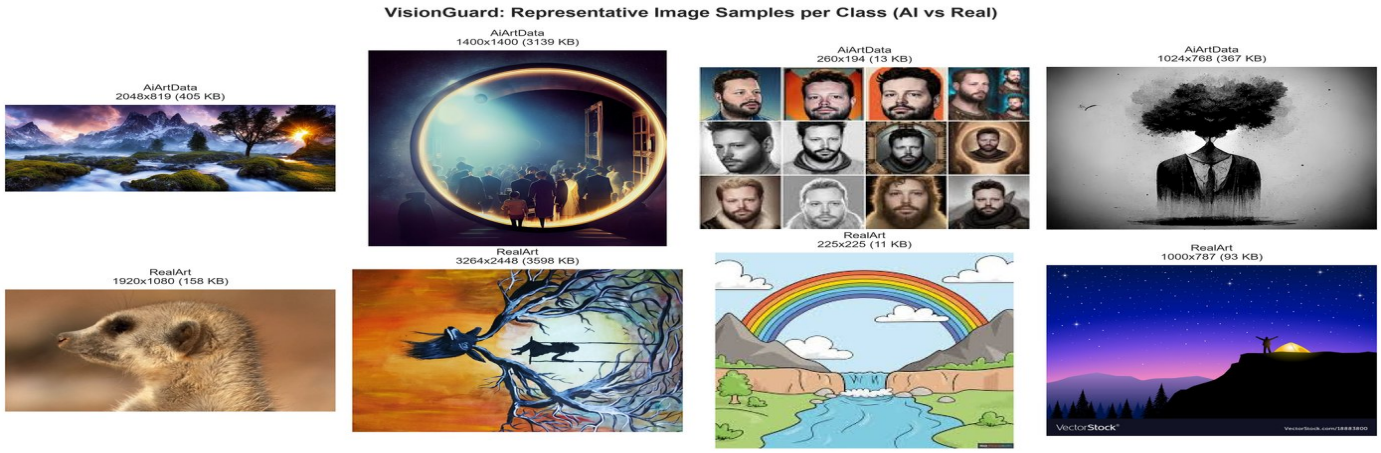


***Figure 1:*** *Image samples from AiArtData and RealArt with dimensions and file footprints.*

**DVC Data Versioning:**
• *Toy Verification:* In scripts/task1/run_toy_dvc.py, I initialized Git and DVC, tracked version 1 of a CSV dataset, pushed it to a local remote, updated to version 2, and demonstrated checking out version 1 using git checkout and dvc checkout. Deleting the local file and restoring it with dvc pull worked as expected.
• *Dataset Tracking:* The entire raw dataset directory was tracked with dvc add data/raw, generating data/raw.dvc (MD5: 8f23d6dc9b06d363c3981642c50856c0.dir, 503.78MB, 975 files). The DVC remote was configured on DagsHub (https://dagshub.com/Shayan-Qadir/23L0790_A2_Ops.dvc) and pushed using an authenticated token.
• *Why DVC instead of Git:* Git uses delta compression designed for text source code. Tracking compressed binary images directly in Git rapidly inflates repository size and exceeds GitHub's 100MB limit. DVC decouples storage from version control: Git tracks the small YAML pointer file while large binaries stay on cloud storage.

## Task 2: Model Development & Pipeline

**Input Pipeline (src/models/dataset_pipeline.py):** The dataset was split into 70% Train (681 images), 15% Validation (146 images), and 15% Test (147 images) using stratified sampling with seed 42. The tf.data pipeline performs: (1) asynchronous byte reading, (2) forced 3-channel RGB conversion (slicing RGBA alpha channels and expanding grayscale channels), (3) bilinear resizing to 224×224, (4) training-only augmentations (horizontal flip, ±10% rotation, ±10% zoom, ±10% contrast), and (5) batching (size 32) with AUTOTUNE prefetching.

**Baseline CNN:** Built from scratch with 3 Conv2D blocks (32, 64, 128 filters with 3×3 kernels), Batch Normalization, and MaxPooling2D. Instead of a Flatten layer (which produces over 1.2M parameters and overfits), GlobalAveragePooling2D was used, reducing parameters to 103,425. The classification head uses Dense(64, ReLU), Dropout(0.4), and Dense(1, Sigmoid). Loss: Binary Crossentropy, Optimizer: Adam (lr=1e-3).

**Transfer Learning (MobileNetV2):** MobileNetV2 pretrained on ImageNet was loaded without top classification layers. Inputs are scaled to $[-1, 1]$ via mobilenet_v2.preprocess_input. Training was conducted in two stages:
• *Stage 1 (Feature Extraction):* Base model was completely frozen. Trained custom classification head (GlobalAveragePooling2D, Dense 128 with ReLU, BatchNorm, Dropout 0.3, Dense 1 Sigmoid) for 3 warmup epochs.
• *Stage 2 (Fine-Tuning):* Unfroze the top 30 layers (Blocks 14 to 16) while keeping all BatchNormalization layers frozen to protect learned statistics. Trained using Adam with a reduced learning rate of 1e-5.

**Training Discipline & Threshold Selection:** All runs used seed 42, EarlyStopping(patience=3, restore_best_weights=True), and ReduceLROnPlateau(factor=0.2, patience=2). The test set was evaluated once at the very end. On the validation set, threshold sweeps from 0.1 to 0.9 were analyzed. Falsely classifying an authentic artist's work as AI (False Positive) harms real creators' reputation, so a balanced threshold of 0.50 was chosen.

### Task 2.6: Error Analysis (10 Misclassified Images):

I isolated 10 misclassified test images and generated an error analysis grid (reports/figures/models/error_analysis_grid.png). Two main error patterns emerged: (1) Real images with very clean digital lines, smooth gradients, or high color saturation (like scenery.jpg and railroad-163518_640.jpg) were mistaken for AI because their clean textures mimicked diffusion outputs. (2) AI images that simulated vintage photography or heavy canvas grain (like 196027s_art_of_cow_getting_abducted_by_UFO_in_midwest.jpg) bypassed detection because noise masked generative artifacts.



***Figure 2:*** *Error analysis grid showing 10 misclassified test images with true vs predicted labels.*

### Task 2.7: Hyperparameter Tuning (Keras Tuner):

I ran Keras Tuner RandomSearch on both a toy MLP (Fashion-MNIST, 6 trials) and MobileNetV2 (8 trials) across learning rates (1e-3, 1e-4, 1e-5), dropout rates (0.2, 0.3, 0.5), and number of unfrozen layers (15, 30, 45).

| Trial Rank | Learning Rate | Dropout | Unfrozen Layers | Val Accuracy |
|---|---|---|---|---|
| 1 (Best) | 0.0001 | 0.5 | 15 | **78.08%** |
| 2 | 0.0001 | 0.3 | 45 | 67.12% |
| 3 | 0.00001 | 0.5 | 45 | 62.33% |
| 4 | 0.00001 | 0.2 | 15 | 59.59% |
| 5 | 0.00001 | 0.5 | 30 | 58.22% |
| 6 | 0.00001 | 0.3 | 15 | 56.16% |
| 7 | 0.001 | 0.3 | 30 | 44.52% |
| 8 | 0.001 | 0.2 | 45 | 44.52% |

*Tuning Outcome:* Learning rate was the most decisive factor. Rates of 1e-3 degraded pretrained weights (val accuracy dropped to 44.5%), while 1e-4 provided steady convergence. Furthermore, tuning solely on validation data risks overfitting to validation quirks, which confirms why final evaluation must be on an isolated test set.

### Bonus Task: Grad-CAM Interpretability Analysis:

Using MobileNetV2's final convolutional layer (Conv_1), I computed Grad-CAM activation heatmaps for 4 correct and 4 misclassified test images (reports/figures/models/gradcam_analysis.png). For correct AI predictions, heatmaps show focus on unnatural facial smoothness and background blur gradients. For incorrect predictions, the model often got distracted by peripheral borders, watermarks, or flat saturated color blocks.

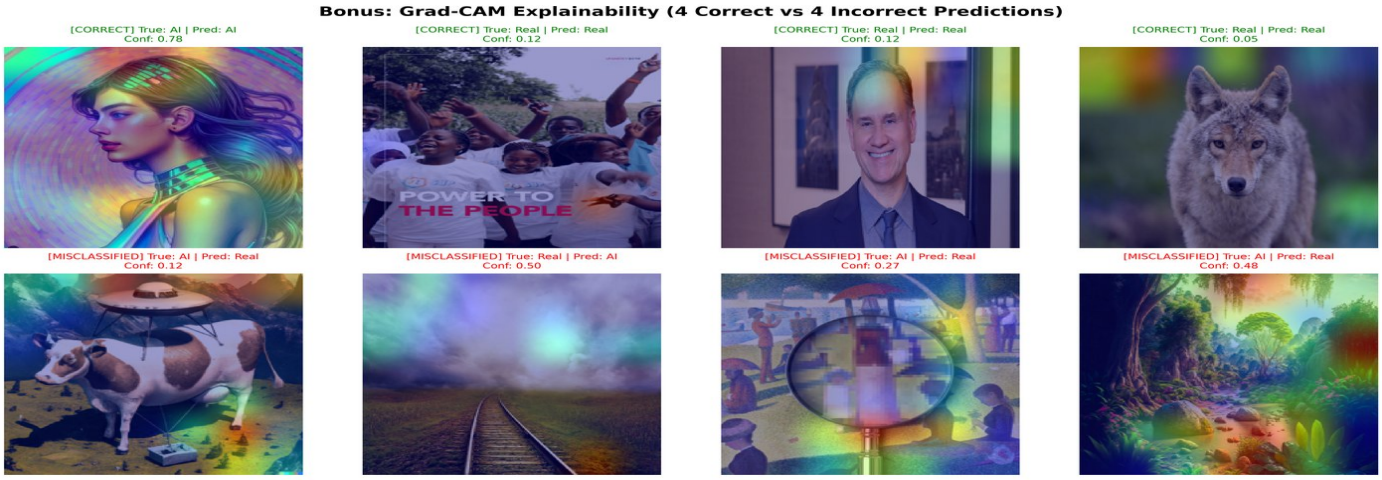



***Figure 3:*** *Grad-CAM heatmaps for correct predictions (top) and misclassified predictions (bottom).*

## Task 3: Experiment Tracking & Model Registry (MLflow & DagsHub)

**MLflow Tracking:** Experiments were tracked on DagsHub MLflow (https://dagshub.com/Shayan-Qadir/23L0790_A2_Ops.mlflow).
• *Manual Logging vs Autologging:* In Run 1, I used manual logging (mlflow.log_metric, mlflow.log_figure) which gave full control over custom metrics (F1, threshold, test plots) without extra overhead. In Run 2, I tested mlflow.tensorflow.autolog(), which captured epoch metrics and computational graphs automatically, but created larger log artifacts.
• *6-Run Comparison:* I tracked a 6-run comparison suite under the experiment VisionGuard_Comparison_Suite.

| Run Name | Model Type | Threshold | Accuracy | Precision | Recall | F1-Score | ROC-AUC |
|---|---|---|---|---|---|---|---|
| 1_Baseline_CNN_Manual | Custom CNN (Aug) | 0.44 | 55.78% | 56.45% | 86.42% | 0.6829 | 0.5585 |
| 2_Baseline_CNN_Autolog | Custom CNN (Auto) | 0.41 | 57.82% | 58.72% | 79.01% | 0.6737 | 0.5556 |
| 3_Baseline_CNN_NoAug | Custom CNN (NoAug) | 0.45 | 57.14% | 56.82% | 92.59% | 0.7042 | 0.5470 |
| 4_MobileNetV2_Frozen | Transfer (Frozen) | 0.47 | 78.91% | 82.05% | 79.01% | 0.8050 | 0.8644 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5_MobileNetV2_FineTuned | Transfer (Fine-Tuned) | 0.27 | **76.87%** | **73.74%** | **90.12%** | **0.8111** | **0.8427** |
| 6_MobileNetV2_Dropout05 | Transfer (Drop 0.5) | 0.37 | 76.87% | 74.23% | 88.89% | 0.8090 | 0.8674 |

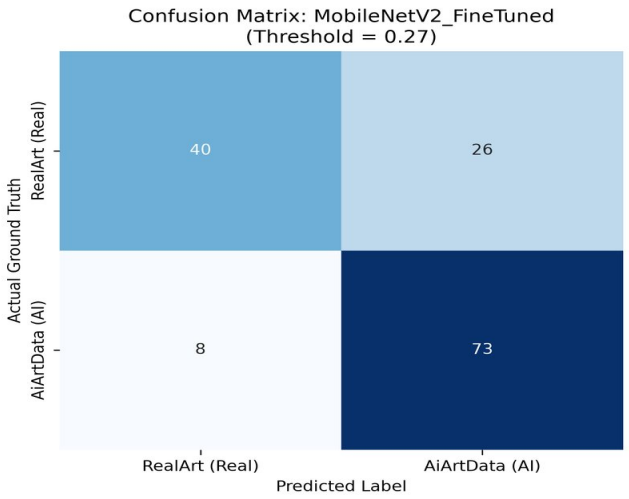



***Figure 4a:*** *Champion Confusion Matrix (Test Set)*

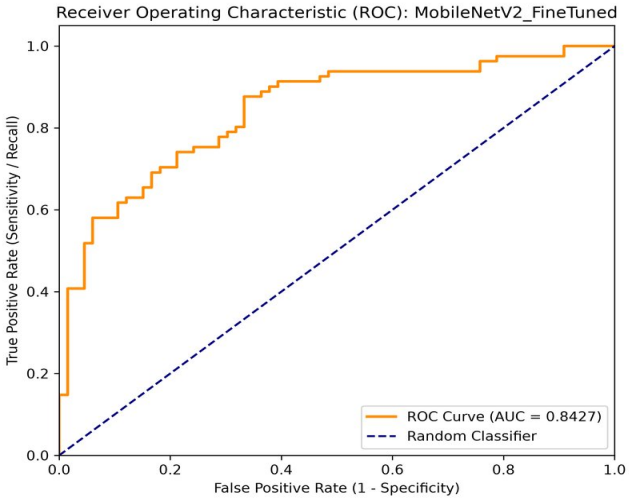



***Figure 4b:*** *Champion ROC Curve (AUC = 0.8427)*

**Champion Selection & Model Registry:**
Run 5 (MobileNetV2 Fine-Tuned) was selected as Champion. It achieved the highest Recall on AI images (90.12%) and the highest F1-score (0.8111), substantially outperforming the baseline CNNs which struggled with low accuracy (~56%) on raw pixel inputs. I registered the model in the DagsHub MLflow Model Registry under the name VisionGuard. Version 2 was assigned the alias @champion. I wrote scripts/task3/load_champion.py which loads the champion model by alias (models:/VisionGuard@champion) and performs inference on test samples.

## Tasks 4 & 5: Flask Application & Database Audit Trail

**Inference Engine Design (src/app/inference.py):** Implemented as a singleton pattern. Model weights are loaded once into memory when the server starts, rather than reloading on every request. Input images are decoded directly from memory bytes using PIL, resized to 224×224, and passed to the model without writing any files to disk. This gives fast response times (~30ms local CPU, ~14ms on Vercel).
**Flask App & Validation:** The UI (templates/index.html) supports drag-and-drop file upload. Uploads are secured using secure_filename, a strict 5MB upload limit via MAX_CONTENT_LENGTH, and MIME type verification. Custom error templates were implemented for 400, 413, and 500 errors.
**Database Audit Trail (Task 5):** Using Flask-SQLAlchemy, every prediction is logged to a prediction_logs table. Fields stored include timestamp (UTC), original filename, verdict (AI or Real), confidence score, raw probability, active threshold, and processing latency in milliseconds. A dedicated administrative route (/history) displays all logged predictions in a searchable table. The app supports both SQLite (locally and on Vercel via /tmp/visionguard.db) and PostgreSQL via the DATABASE_URL environment variable.
**Automated Tests:** I wrote 8 automated pytest unit tests in tests/test_app.py and tests/test_pipeline.py, verifying model inference, route responses (/, /predict, /history, /health, /api/predict), 5MB file size limits, and database writes. All 8 tests pass.

## Task 6: Docker Containerization, Compose & CI

**Dockerfile:** Built using python:3.11-slim. By using CPU-only TensorFlow in requirements_docker.txt, the container image was reduced from over 2.5GB to ~540MB. For security, an unprivileged user appuser (UID 10001) was created so the application does not run as root. A container health check probes curl http://localhost:5000/health every 30 seconds.
**Docker Compose Stack:** Coordinates two services on a private bridge network (visionguard_network):
• web: Flask container exposing port 5000 to the host.
• db: PostgreSQL 15 container using a named volume db_data for persistent storage. Its port (5432) is closed to the outside host and accessible only to the web container. The web container uses condition: service_healthy to ensure PostgreSQL is ready before starting.
**GitHub Actions CI (.github/workflows/ci.yml):** Configured to trigger on every push and PR to main. The pipeline runs: (1) Flake8 linting, (2) automated Pytest testing across Python 3.10 and 3.11, and (3) a Docker build check to verify container builds cleanly.

## Tasks 7 & 8: Cloud Deployment & External API Integration

**Pathway B: Vercel Serverless Deployment (Primary):**

• *Overcoming the 250MB Function Limit:* Standard TensorFlow exceeds Vercel's 250MB uncompressed limit. To solve this, I converted the champion Keras model (23.7MB) into a FlatBuffer TFLite model (models/champion/model.tflite, 9.1MB). In production, I used Google's ai-edge-litert runtime (~20MB), bringing the total function package down to ~45MB.

• *Read-Only Filesystem & Database:* Vercel functions run in a read-only container. Image processing is done entirely in memory using io.BytesIO with zero disk writes. For database logging, the SQLite path is routed to /tmp/visionguard.db (writable scratch storage on Vercel) or external PostgreSQL.

• *Live Deployment URLs:*

– Web UI: https://23l-0790a2mlops.vercel.app

– REST API: https://23l-0790a2mlops.vercel.app/api/predict

**Pathway A: Oracle Cloud Infrastructure VM (Alternative Scripted):**

• I also scripted full deployment for an Always-Free Ubuntu VM on Oracle Cloud (scripts/task7/deploy_oracle_vm.sh) running Docker Compose (Flask + PostgreSQL). The script addresses OCI's dual firewall by setting up Security List Ingress rules and opening host firewall rules via sudo iptables -I INPUT 6 -p tcp --dport 5000 -j ACCEPT.

**Task 8: External API Verification:**

I wrote scripts/task7/test_cloud_api.py to test the public cloud endpoint externally. The script sends an image via multipart form POST, handles bypass tokens if protection is active, verifies HTTP 200, checks all required JSON fields (prediction, confidence, raw_probability, threshold, latency_ms), and logs roundtrip latency.